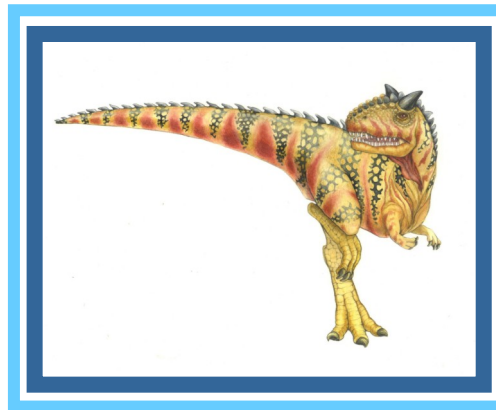
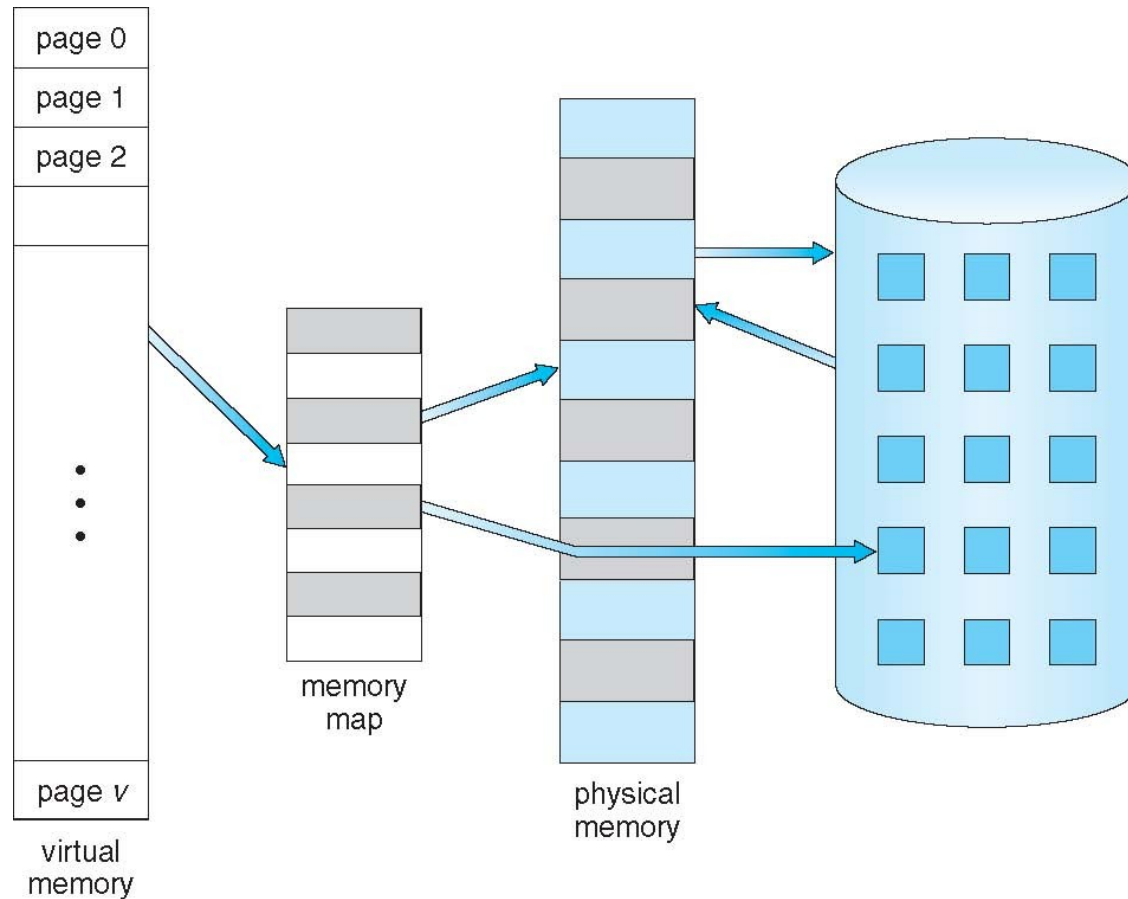


Chapter 8: Memory Management





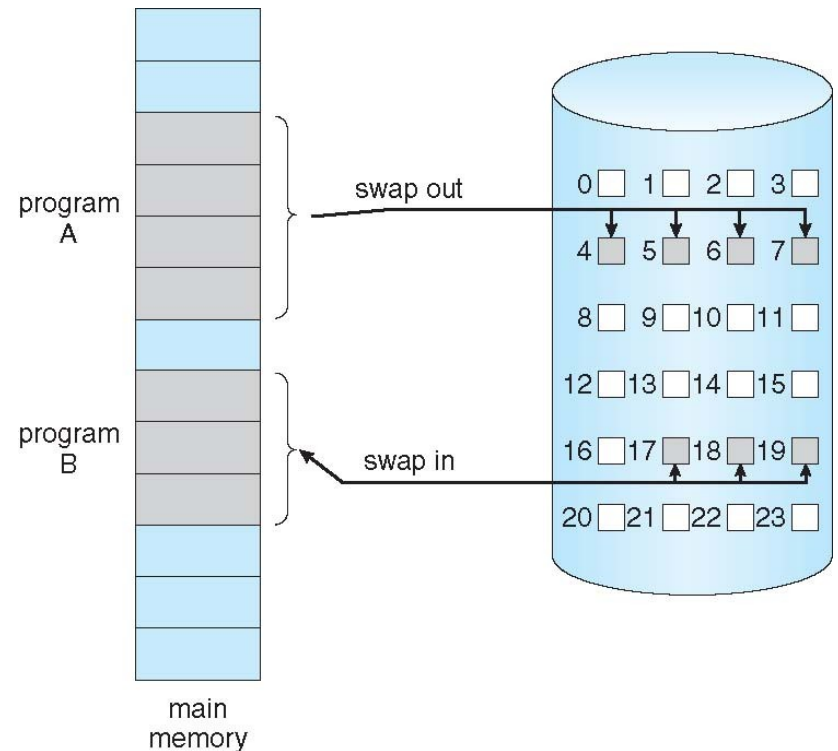
Virtual Memory That is Larger Than Physical Memory





Demand Paging

- ❑ Could bring entire process into memory at load time
- ❑ Or bring a page into memory only when it is needed – Demand Paging
 - ❑ Less I/O needed, no unnecessary I/O
 - ❑ Less memory needed
 - ❑ Faster response
 - ❑ More users
- ❑ Similar to paging system with swapping (diagram on right)
- ❑ Page is needed $\Rightarrow$ reference to it
 - ❑ invalid reference $\Rightarrow$ abort
 - ❑ not-in-memory $\Rightarrow$ bring to memory
- ❑ **Lazy swapper** – never swaps a page into memory unless page will be needed
 - ❑ Swapper that deals with pages is a **pager**





Basic Concepts

- Instead, pager brings in only those pages that are currently demanded into memory
- If pages needed are already **memory resident (Valid)**
 - No difference from non demand-paging
- If page needed and not memory resident **(Invalid)**
 - Need to detect and load the page into memory from storage
 - ▶ Without changing program behavior
 - ▶ Without programmer needing to change code





Valid-Invalid Bit

- A new MMU functionality with valid-invalid bit.
- With each page table entry a valid–invalid bit is associated (**v** $\Rightarrow$ in-memory – **memory resident**, **i** $\Rightarrow$ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:

Frame #	valid-invalid bit
	v
	v
	v
	i
...	
	i
	i

page table

- During MMU address translation entry is **i** $\Rightarrow$ page fault

: in page table





Page Table When Some Pages Are Not in Main Memory

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

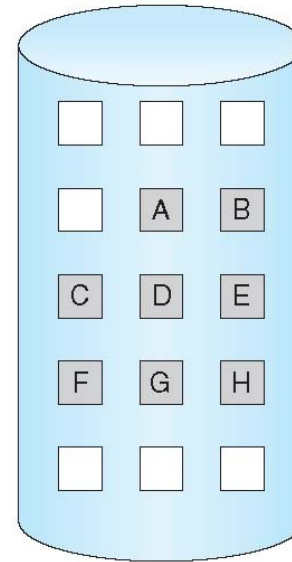
logical
memory

valid-invalid bit		
frame		
0	4	v
1		i
2	6	v
3		i
4		i
5	9	v
6		i
7		i

page table

0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15

physical memory





Page Fault

- If there is a reference to a page, first reference to that page will trap to operating system:

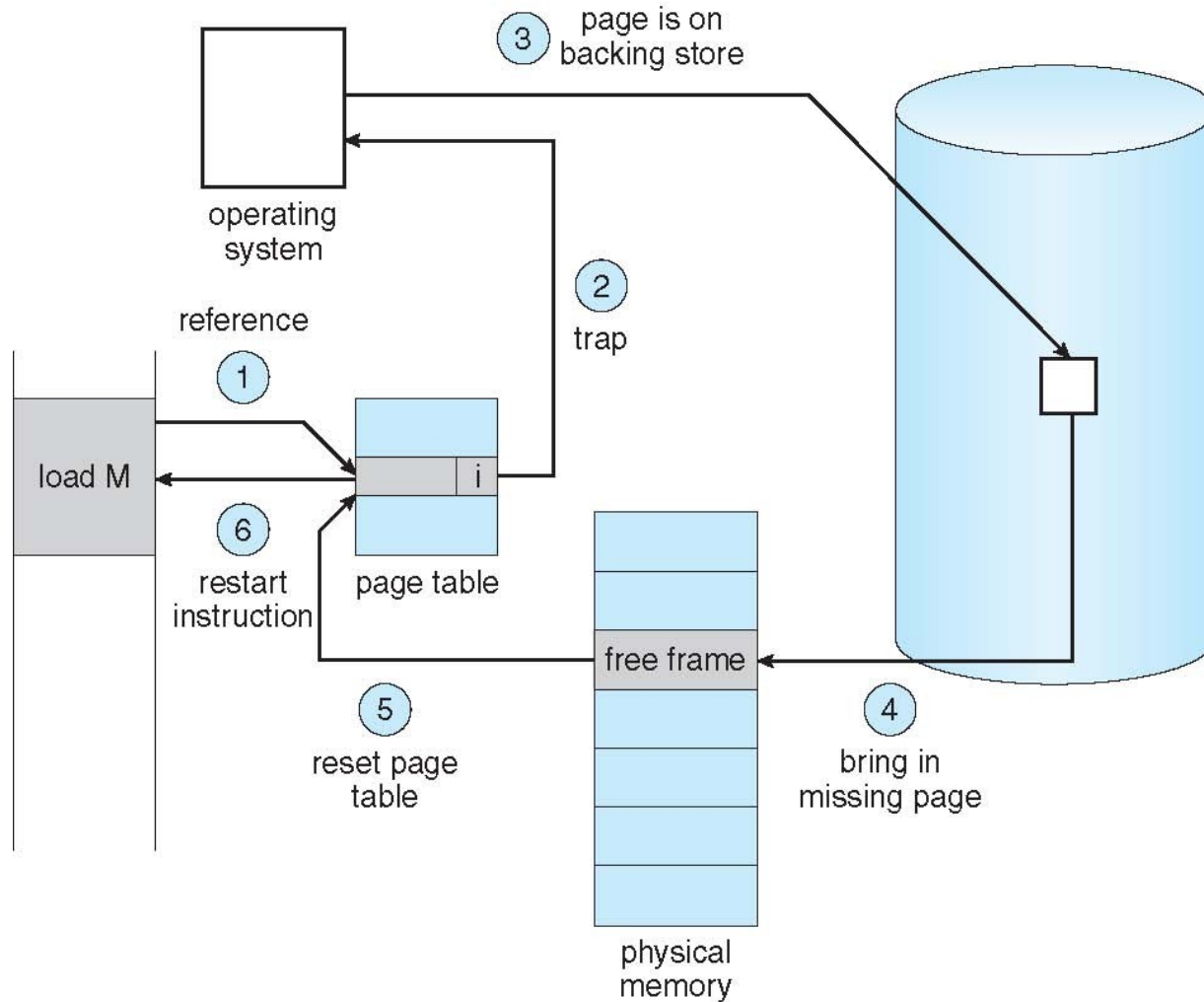
page fault

1. Operating system looks at another table to decide:
 - Invalid reference $\Rightarrow$ abort
 - Just not in memory
2. Find free frame
3. Swap page into frame via scheduled disk operation
4. Reset tables to indicate page now in memory
Set validation bit = **v**
5. Restart the instruction that caused the page fault





Steps in Handling a Page Fault





What Happens if There is no Free Frame?

- Used up by process pages
- Also in demand from the kernel, I/O buffers, etc
- How much to allocate to each?
- Page replacement – find some page in memory, but not really in use, page it out
 - Algorithm – terminate? swap out? replace the page?
 - Performance – want an algorithm which will result in minimum number of page faults
- Same page may be brought into memory several times





Page Replacement

- Prevent **over-allocation** of memory by modifying page-fault service routine to include page replacement
- Use **modify (dirty) bit** to reduce overhead of page transfers – only modified pages are written to disk
- Page replacement completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory
- ***Find a victim frame to replace.***





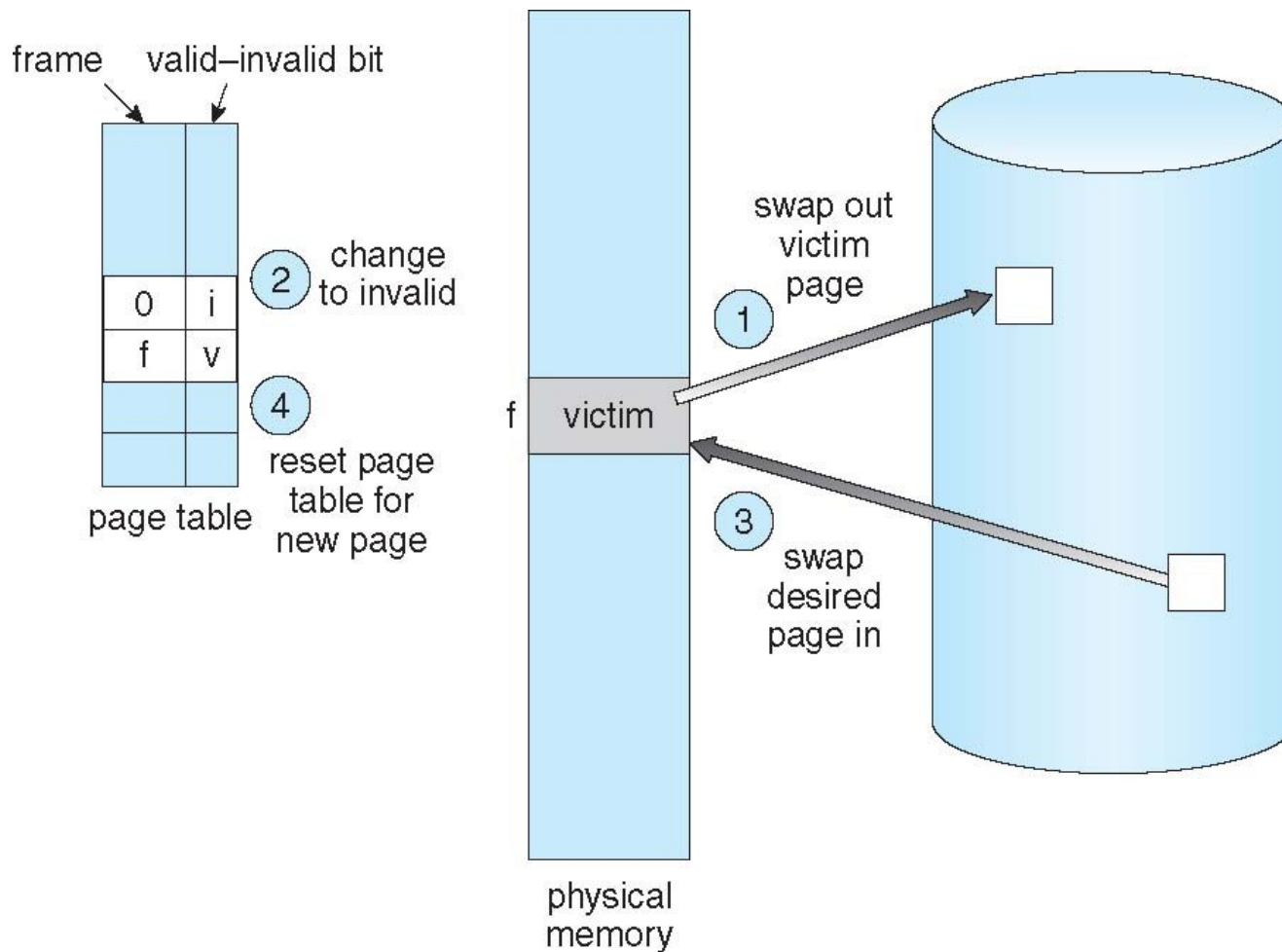
Basic Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame:
 - If there is a free frame, use it
 - If there is no free frame, use a page replacement algorithm to select a **victim frame**
 - Write victim frame to disk if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap





Page Replacement





Page and Frame Replacement Algorithms

- **Frame-allocation algorithm** determines
 - How many frames to give each process
 - Which frames to replace
- **Page-replacement algorithm**
 - Want **lowest page-fault rate** on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
 - String is just page numbers, not full addresses
 - Repeated access to the same page does not cause a page fault
 - Results depend on number of frames available
- In all our examples, the **reference string** of referenced page numbers is

7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1





First-In-First-Out (FIFO) Algorithm

- Reference string: **7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1**
- 3 frames (3 pages can be in memory at a time per process)

reference string

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2	2	2	4	4	4	0	0	0	0	0	0	7	7	7
	0	0	0	3	3	3	2	2	2	1	1				1	0	0
		1	1	1	0	0	0	3	3	3	2				2	2	1

page frames

15 page faults

- How to track ages of pages?
 - Just use a FIFO queue





Optimal Algorithm

- Replace page that will not be used for longest period of time
 - 9 is optimal for the example (**OPT**)
- How do you know this?
 - Can't read the future
- Used for measuring how well your algorithm performs

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2		2		2					7
	0	0	0		0		0		0		0					0
		1	1		3		4		3		1					1

page frames





Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

page frames

- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?





Use Of A Stack to Record Most Recent Page References

reference string

4 7 0 7 1 0 1 2 1 2 7 1 2

2
1
0
7
4

stack
before
a

7
2
1
0
4

stack
after
b

↑
a

↑
b





In Class Practice

- Consider a paged virtual memory system, in which a process is allocated with a 3 frames window. Assume the memory reference string is 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5.
- Using LRU page-replacement algorithm, how many page faults in total?
- 10





Structure of the Page Table

- ❑ Memory structures for paging can get huge using straightforward methods
 - ❑ Consider a 32-bit logical address space as on modern computers
 - ❑ Page size of 4 KB (2^{12})
 - ❑ Page table would have 1 million entries ($2^{32} / 2^{12}$)
 - ❑ If each entry is 4 bytes -> 4 MB of physical address space / memory for page table alone
 - ▶ That amount of memory used to cost a lot
 - ▶ Don't want to allocate that contiguously in main memory
- ❑ Hierarchical Paging





Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table





Two-Level Paging Example

- A logical address (on 32-bit machine with 1K page size) is divided into:
 - a page number consisting of 22 bits
 - a page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
 - a 12-bit page number – p_1
 - a 10-bit page offset – p_2
- Thus, a logical address is as follows:

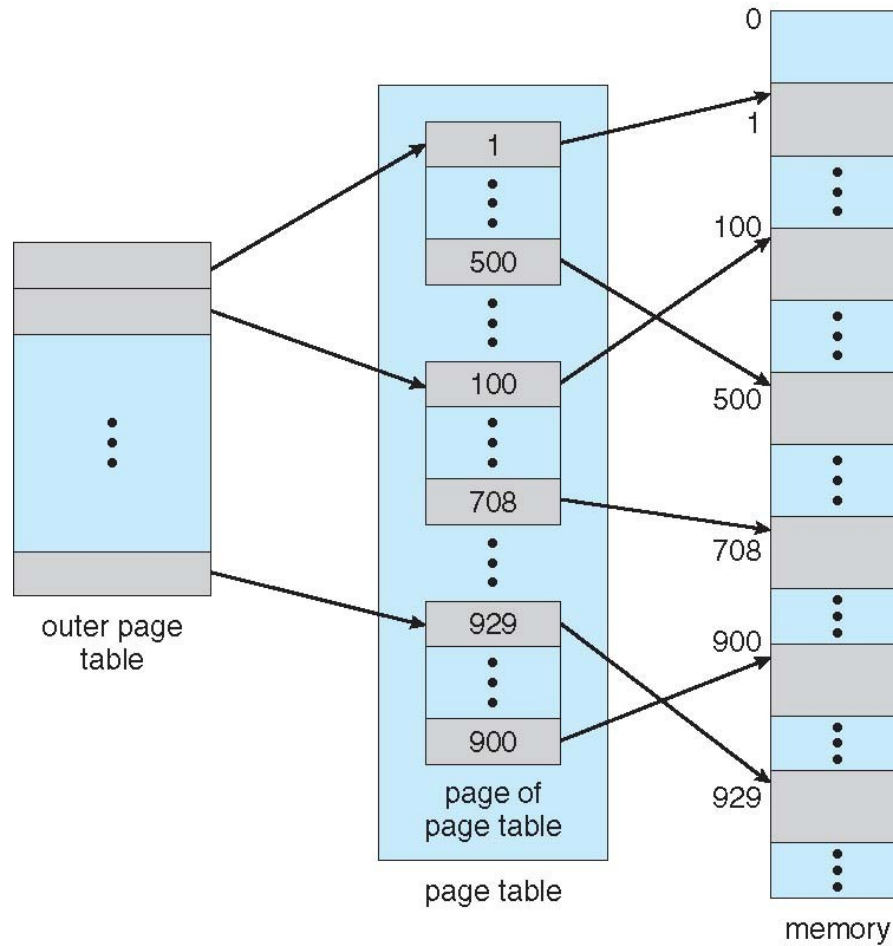
page number		page offset
p_1	p_2	d
12	10	10

- where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table
- If each entry is 1B, then two level table size $2^{12} + 2^{10} = 5KB$, compared to single level table size $\frac{2^{32}}{2^{10}} = 2^{22} = 4MB$



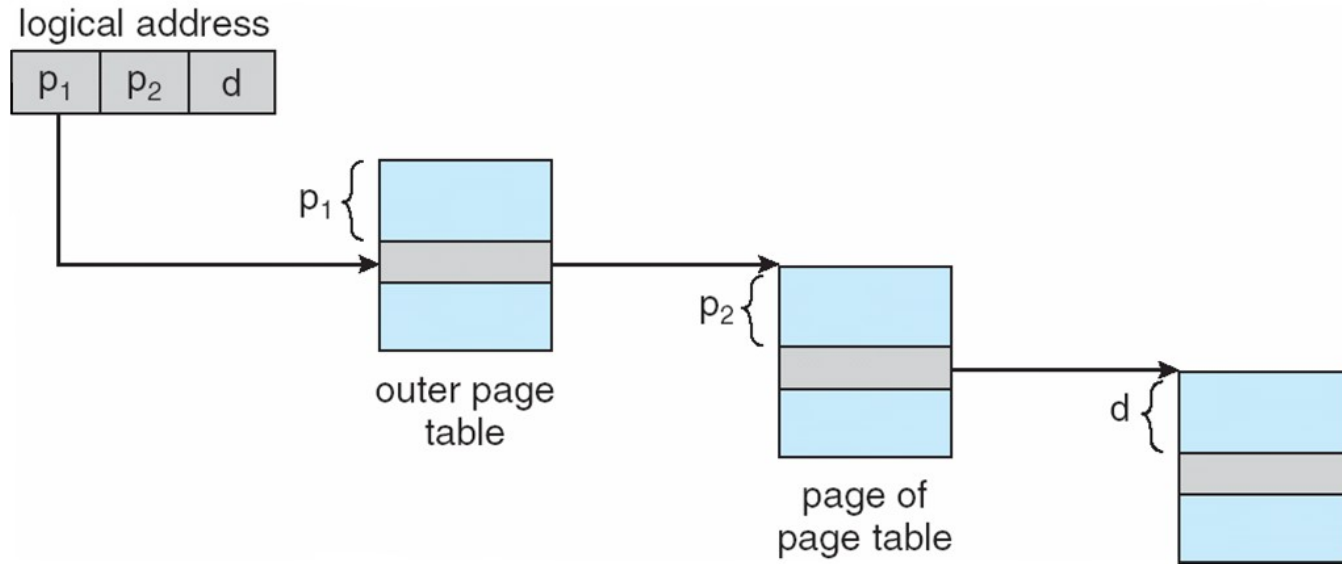


Two-Level Page-Table Scheme





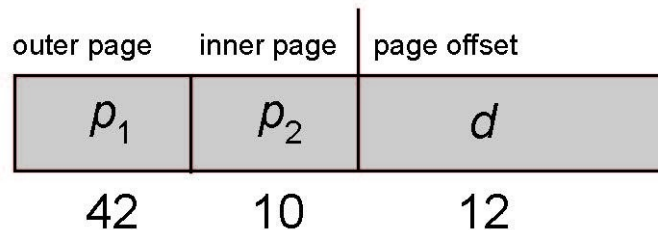
Address-Translation Scheme





64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
 - Then page table has 2^{52} entries
 - If two level scheme, inner page tables could be 2^{10} 4-byte entries
 - Address would look like



- Outer page table has 2^{42} entries or 2^{44} bytes
- One solution is to add a 2^{nd} outer page table
- But in the following example the 2^{nd} outer page table is still 2^{34} bytes in size
 - ▶ And possibly 4 memory access to get to one physical memory location





Three-level Paging Scheme

outer page	inner page	offset
p_1	p_2	d
42	10	12

2nd outer page	outer page	inner page	offset
p_1	p_2	p_3	d
32	10	10	12

- Single level table size: $\frac{2^{64}}{2^{12}} = 2^{52} = 4PB$
- Two level table size: $2^{42} + 2^{10} = 4TB$
- Three level table size: $2^{32} + 2^{10} + 2^{10} = 4GB$





Review of Memory Management

- How to defragmentation?
 - Segmentation
 - ▶ Split the request into smaller modules – segmentation, so that they are flexible to use the fragmented memory
 - Paging
 - ▶ CPU see a set of continuous **logical address**
 - ▶ The **physical address** in the memory can be discontinuous
 - ▶ Page table maps logical address to physical address
 - Hierarchical page table addresses the scalability issue
 - Swap and backing store
 - ▶ Increase the size of usable physical memory
 - ▶ Page fault
 - ▶ Page replacement

